LingoLive — Find your next words

AI Builder Cup 2026 · Prepared for Ashish T. Vasant · 17 October 2026

Theme: Media, Content & Digital Experiences. **Category:** Education.

Problem statement: Help adult language learners understand everyday spoken conversations and formulate a useful next reply, with explanations they can read and learning they can revisit.

Live prototype: https://lingolive-358650584819.us-central1.run.app

Source: https://github.com/U4AR/LingoLive

The problem

Knowing individual words does not always help someone respond in a real conversation. Travellers, newcomers and people working in another language may understand part of a sentence but struggle with the next turn. A separate translation screen, pronunciation tool and vocabulary notebook interrupt the exchange. A suggested reply can also be wrong when it forgets what the learner already wanted to do.

LingoLive turns the conversation itself into a short lesson: what was said, what the words mean, and three things the learner could say next. Its intended benefit is more accessible, contextual language practice. Learning gains and real-world accessibility outcomes have not yet been measured.

What the prototype does

The learner chooses a language they understand and one they want to learn. A first-use tutorial demonstrates word meanings and pronunciation using a clearly labelled prepared example. It makes no paid model call and starts no microphone or location request. The main screen then presents one large listening button; typed and pasted phrases are also supported.

Google transcribes speech across languages. Finalized speech becomes a lesson when the selected learning language is present. Other finalized speech remains available as a transcript. Mixed-language phrases can be handled, although language detection is an estimate. Interim captions remain temporary.

The lesson shows the source, a translation and three short suggested replies. Tapping a source or reply word opens its contextual meaning and pronunciation in a bubble. Original-script and pronunciation views use the same stored word data. Hearing or copying a reply, switching views and opening a word meaning do not require another model call. Voice playback depends on device voices.

Saved lessons can be searched, reopened and deleted on the same browser. The interface keeps settings and detailed explanations behind optional controls. The Android companion exists separately; this submission demonstrates the current deployed web application.

Replies that follow the conversation

The learner can state a goal, such as wanting tea to take away. Later replies receive that goal alongside the current utterance, recent turns, the previous suggestions, a short scene summary, learner notes and previously seen vocabulary. When no goal is supplied, the model estimates the immediate purpose and offers plausible replies or clarification. Explicit goals override tentative inferences.

Suggested text is kept separate from confirmed speech. The learner can mark “I said this” to record a reply without another generation request. Model-derived speaker labels are estimates from the conversation, not verified identity or reliable diarization. Seen vocabulary is not presented as mastered vocabulary.

Memory is bounded: at most eight recent turns, twelve seen words, editable notes of 800 characters and a scene that expires after 90 minutes of inactivity. New conversation resets the scene while preserving saved learning. Local learner profiles separate notes and history; they are not authenticated accounts, and anyone using the same browser can switch them.

Meaningful use of Google AI and Google Cloud

A single Cloud Run service serves the web interface, FastAPI lesson API and authenticated speech WebSocket from one HTTPS origin. Browser AudioWorklet code resamples microphone input to mono 16 kHz PCM. **Gemini 3.5 Transcribe Live** provides streaming transcription. The backend uses **Gemini 3.1 Flash-Lite** with minimal thinking for two parallel tasks: source understanding and reply generation.

Both tasks receive the current transcript and incoming learner/conversation context. The source task returns translation, word meanings, readings and scene estimates. The reply task returns three short replies with translations, meanings and readings. The backend validates and merges both into the existing saved lesson format. Compact nonempty word triples reduce generated output; spacing and punctuation are restored deterministically from the exact source. Missing, changed or reordered lexical content and missing definitions fail validation.

There is no third summarization call or automatic retry. If a branch fails, its sibling is cancelled and the transcript remains available for manual retry. Replies use incoming context immediately; newly inferred source/audio context can influence the next turn, rather than delaying the reply branch.

The host uses Vertex AI with a dedicated Cloud Run service account, without a Google API key in the browser or recommended host runtime. Alternatively, users can connect their own Google AI Studio key. Personal-key mode uses that key for both speech and lessons and never falls back to host credentials.

Optional setting awareness

Settings let the learner choose a broad scene such as home, work, a café or travelling. Approximate location and recent sound are optional and off by default. Location is rounded to two decimal coordinates, kept in RAM and expires after five minutes. There is no paid Maps API dependency. Recent sound is limited to four seconds and sent as audio only to the source-understanding task. Neither signal is saved into lesson history. These signals help estimate a setting; they do not verify a venue or establish sensitive personal facts.

Performance: measured improvement, with limits

The earlier one-call Gemini 3.8 Flash deployment averaged **9.26 seconds** over ten complete lesson requests. A direct same-model comparison found that simply splitting the original output into two parallel requests was insufficient: three prepared cases averaged 6.71 seconds with one call and 9.55 seconds with two original-format calls. Compact parallel output on 3.8 averaged 3.61 seconds.

The final design combines parallel requests, Flash-Lite, concise output and warm HTTP connection reuse. It averaged **1.79 seconds over ten complete, validated hosted lessons with zero failures**. The six sequential Japanese/Hindi/Spanish requests averaged 1.75 seconds. Two simultaneous independent sessions completed in 1.65 and 1.71 seconds. A separate functional context/audio check took 2.34 seconds; including it gives an eleven-lesson mean of 1.84 seconds.

These are small warm-path samples, not a promise for every language, long input, mobile network or cold start. The synthesized speech test took 3.54 seconds from the end of audio to the complete lesson, including 1.11 seconds of test-client handoff. Thus the measured two-second lesson average does not describe the whole speech pipeline. Physical-phone latency remains unverified. Slower and failed experiments are retained in the repository rather than excluded from the comparison.

Privacy, authentication and cost controls

Host-funded usage requires a private access code. Browser sessions use signed, opaque, Secure, HttpOnly, SameSite=Strict cookies and expire after one hour. Cookie writes and WebSocket connections require the app's own origin. Google account sign-in is optional in the code but disabled in this deployment.

Personal API keys stay in server RAM for at most one hour. They are not stored in cookies, IndexedDB, LocalStorage, source code or a transcript database. Disconnect revokes the session; restart or scale-to-zero forgets personal keys. Users must trust the host that receives their key. Google bills the personal key's project; the host still pays relay infrastructure costs.

Lessons and vocabulary stay in browser IndexedDB; notes and settings stay in LocalStorage. The app does not create a server-side conversation history database or save raw audio recordings. Selected transcript/context necessarily goes to Google to provide the requested AI features. Clearing browser data deletes local learning. There is no account sync or cross-device history.

Cloud Run has zero minimum and one maximum instance, one CPU, 512 MiB and request-based billing. The backend permits two simultaneous lessons and two active listeners. Raw model-call allowances remain 24/minute, 360/hour and 400/day; two calls per parallel lesson consume two units, allowing 12/180/200 lessons. The two branches share a combined maximum of 4,096 output tokens. Host and personal-key groups have separate shared model allowances. Audio connection time, including silence, is limited to 40 combined minutes/hour and 60/day. Authentication, input bounds, audio pacing and an emergency AI pause run before paid operations. In-memory counters reset after restarts and are usage guards, not a hard dollar cap.

Validation and current readiness

93 backend tests and **57 real Chromium browser checks** passed, along with JavaScript source/word/audio/context checks. Local tests use explicit Google, microphone and location fixtures. Separate hosted checks used real Google transcription and lessons, validated exact text/readings/meanings, checked access controls and proved invalid personal keys do not produce a host-funded fallback. Valid personal-key inference and physical-phone capture remain unverified.

Production revision `lingolive-00010-naj` serves web 1.5 on Cloud Run. The same revision was checked through a temporary tag before promotion; public health, assets and routing were verified afterward. The normal persistent Cloud SDK operator login is separate from the runtime service account. No deployment credentials are included in the repository or demonstration.

Differentiation and growth

The prototype brings together live conversation, contextual next-turn replies, tappable words, pronunciation and local learning memory in one small interface. Its engineering focus is a responsive complete lesson, rather than requiring a new paid call for each word or pronunciation toggle. It does not claim to be the first translation or conversational tutoring product.

For wider adoption, the next steps are representative learner studies, mobile and accessibility testing, language-quality evaluation, durable per-user quotas, stronger identity and a deliberate design for cross-instance key sessions. The current maximum-one-instance setting is a prototype cost choice, not a production scaling claim. Learning retention, task completion, reply usefulness, word-meaning accuracy and real-device latency should be measured before claiming educational outcomes.

Submission materials and references

- Proposal PDF: `LingoLive-Proposal.pdf` in this directory.
- Prepared form responses: `ENTRY.md` and `submission.json`.
- Demo narration, captions and provenance: `demo-scenes.json`, `DEMO.md` and `validation/submission-demo.json`.
- Technical detail: `MEMORY.md`, `WEB-DEPLOY.md`, `VALIDATION.md` and `docs/LATENCY.md`.
- Official requirements checked October 17, 2026: https://aibuildercup.com/themes.html and https://aibuildercup.com/Faqs.html.
- Registered participants access their dashboard at https://hack2skill.com/event/aibuildercup2026/registration?isExternalLinkRedirection=true.

The official site lists October 18, 2026 as the submission deadline; a precise time zone/cutoff was not established. The FAQ requires a public source repository, a video under three minutes on YouTube/Vimeo or public Google Drive, and an eligible registered team of two to four. This document does not assert eligibility or an accepted entry. Portal receipt, team details and the required video-host link must be verified at actual submission.